

Claude Code — a working day

A study guide for an agentic coding tool: what it changes about the job, what it does not, and where the leverage actually is.

Each section states an idea, makes the argument for it, and points at the lab that tests it. The labs live in [cc-exercises/](#); each has its own brief.

On sources. This material is original. It is informed by Ken Kousen's *Claude Code: Up and Running* (O'Reilly), and none of its text or prompts is reproduced here — where a move is his, the section cites the chapter and describes what the prompt has to accomplish so you can write your own. The experiment in §4, the fixtures, and the verifier are this repo's.

0. Three scales, and why the middle one matters

A useful way to size work before you start:

- a **doghouse** — one file, no tests, no users but you. Mistakes cost minutes.
- a **cabin** — a few modules, real tests, someone else will read it. Mistakes cost an afternoon.
- a **skyscraper** — a system with history, dependencies and downstream consumers. Mistakes cost a quarter.

Kousen opens with this framing (ch. 1) and it is worth stealing, because the interesting thing is what it predicts: an agentic tool is spectacular at doghouses, genuinely useful at cabins, and needs the most supervision exactly where the stakes are highest. Your working life is mostly cabins and skyscrapers. **A demo is a doghouse. Judge the tool on the cabin.**

That is also why labs 01 through 07 build one small app and then keep going with it. The first hour is the doghouse; everything after is what happens when the same code has to survive a second week.

***The model is a knob.** Speed and depth are a setting, not an identity — reach for a faster model on mechanical work and a deeper one on design. Most people leave the knob where it was and then form an opinion about "the tool".*

1. First contact — the part that is genuinely new

Lab 01. Build something that runs, from nothing, in one session.

What is new is not code completion. It is that the tool reads your files, decides which ones matter, edits several of them, runs the result and reads the error. The loop closes without you in it.

Two habits to form in the first hour:

Ask for a read-only orientation before you ask for a change. "Tell me what this does, change nothing" costs one turn and has zero blast radius. It also tells you whether the thing in front of you understood the repo before it starts rewriting it. (Ch. 2 has his version of that prompt.)

Say how big the first version should be. Left unconstrained, a request for "a small app to practise memorising a poem" comes back with argument parsing, a config file and a package layout. Name the scale: *one file, standard library only, no packaging, no argument parsing*. You can always grow it. The lab's prompt, which is this course's own, is in `exercises/lab02_persistence/README.md`.

And decline the offer at the end. It will propose next steps. Most of them are real. Accepting all of them is how a doghouse becomes an unmaintained cabin in forty minutes.

2. Getting what you asked for

Lab 02. Add persistence to the trainer — badly first, then precisely.

The difference between a vague request and a precise one is not politeness. It is that a vague request leaves the decisions to the model, and you will not be told which decisions those were.

Four things carry almost all the weight:

Put this in	Because
The artifact, by name — a path, a key, a filename	It makes the result inspectable. "Save my place" is checkable only by running the app; "write <code>line_index</code> to <code>~/lyrics-trainer.json</code> " is checkable with <code>cat</code>
A success condition	Otherwise "done" is its opinion. <i>Quit after a few lines, run again, resume at the same line</i> is a test you can perform
A structural fence	<i>Keep the single-file structure, don't extract modules, don't add a dependency.</i> Without it you get the refactor you did not ask for, mixed into the change you did
A clarification pass	Three lines at the end — <i>do you have questions, is there anything I'm missing, are there options I've neglected</i> — and you get the ambiguities before the edit instead of after

The clarification pass is the cheapest thing in this guide. It costs one turn and it routinely surfaces a decision you did not know you were making. Kousen's ch. 3 is built around this comparison; those three lines are his.

Then push back on one decision without asking for a change. *"Why not extract a `save_state()` function?"* is a question, and you get a defence or a concession — either is information. *"Extract a `save_state()` function"* is an instruction, and you get an edit and learn nothing. Keep the two separate; people mix them constantly and then complain the tool is too agreeable.

3. Driving the session

Lab 03. Run `/context` before and after a piece of work. Clear once, on purpose.

Context is the budget you are actually spending. Every file read, every tool result, every one of your own turns stays in it, and turn fifty carries turns one through forty-nine on its back. Two consequences:

Long sessions get worse, not better. Not because the model tires, but because the signal-to-noise ratio of what it is reading drops. Three abandoned approaches and a stack trace from an unrelated bug are still in there competing with your actual question.

Clearing is a tool, not a failure. Finish a piece of work, write down what survived — in `CLAUDE.md`, in a commit message, in a scratch file — and clear. The written artifact is what carries forward; the conversation is not.

Compaction happens automatically when you run out of room, and it is lossy in a specific way: **contracts survive compaction, procedures fray.** A summary keeps "storage is injected, tests never touch the disk" and loses "then we tried three approaches and the second one failed because of X". So write down constraints, not narratives. If something must survive, give it to a file.

Modes. Plan mode proposes and does not edit. Accept-edits stops asking about each write. Auto is the default on paid plans and is not the dangerous one people assume — the boundary is set by your permissions, not by the mode. Know which you are in. The failure is not "the wrong mode", it is not knowing.

Ask for a plan in words before a plan in files. A plan you can read in fifteen seconds is a plan you can reject in fifteen seconds. Then one question, worth memorising: *"what are the alternatives, and why is this one better?"* It is the cheapest defence against a confident plan that is merely the first plan.

4. Tests force the design — sometimes

Lab 04. Ask for tests on a single-file app. Do not ask for a refactor.

Here is the claim, stated so you can disagree with it:

Ask an agentic tool for tests on a messy app and it will not just add tests — it will restructure the code so that tests are possible, and that restructuring is better design you got by asking for something else.

We ran it instead of asserting it. **Nine sessions, three starting points, one prompt that never mentions structure, modules or refactoring.** Read-only, so the tool could only propose.

Starting point	Restructured	Declined
JavaScript — logic inside an inline <code><script></code>	2 of 3	1
Python CLI — everything inside <code>main()</code>	0 of 3	3
Python Flask — <code>requests.get</code> inline in the route handler	0 of 3	3

The claim is language-dependent, and in Python it is false.

The reason is structural, not a model whim. An inline `<script>` exposes nothing — there is nothing to import, so the only path to a test is to make something importable, and the design has to move. **A Python module is always importable.** Globals can be reassigned, `input` can be faked, stdout can be captured, `monkeypatch` is right there. The cheapest path to a test never requires touching the design, and every run found it and reasoned it out loud.

So: **in Python, the tooling will not push back on you.** If you want the seam, ask for it. Three things follow.

1. It will not run the tests unless you tell it to. Do not assume the loop closes itself.

2. "The smallest change" is not a magic word. Both JavaScript runs we kept transcripts for chose Node's built-in test runner — a perfectly reasonable reading of "smallest", and not the structure the lesson needed. Run it twice, get two structures. If your repo needs a particular shape, name the shape.

3. A red test goes green two ways. Fix the code, or quietly weaken the test. Both satisfy "until the suite passes", and a model optimising for a green suite has no intrinsic preference between them. This is why *"it wrote 15 tests"* is a meaningless number: fifteen tests adjusted to fit the code are worse than none, because they are now load-bearing lies. Ask for two lists when it finishes — **what changed** and **what you verified** — and read the second one.

The sharpest finding was about invariants. The one sentence cited most often across the nine runs for *not* touching the design was the prompt's own invariant — *keep the behaviour unchanged*. It is a reasonable sentence, it protects something real, and it blocked the very thing the prompt existed to demonstrate, six times out of nine. **Write your invariants knowing they will be obeyed literally**, and pair every one with a licence to object: *if this constraint makes the task awkward, say so instead of working around it*.

What you actually want here is the seam: storage passed in as a parameter, so a test hands it a fake with `read()` and `write()` and never touches a disk. If the proposal is to monkeypatch `open` instead, that is the wrong half of the fork — it keeps the tangle and adds machinery to work around it.

5. Project memory

Lab 05. Write `CLAUDE.md` by hand, then have it reviewed. Then scope one rule.

`CLAUDE.md` is prepended to every session in the project. That is the whole design, and both of its consequences follow from it.

Write it yourself first. `/init` generates a description of the code — which is the part the agent can already get by reading the code. The value is the part it cannot: *standard library only, only `cli.py` may construct real storage, if `cli.py` grows a conditional, the rule belongs in a module*. Draft those by hand, then ask what a new contributor would get wrong, then cut.

Keep it short. A 300-line memory file is 300 lines of context on every request, including the ones where none of it is relevant. Five sections, under a hundred lines: what this is, architecture, commands, conventions, workflow.

Then scope the rest. Advice about tests is noise while you are editing `storage.py`. A file under `.claude/rules/` with `paths: frontmatter` arrives only when a matching file does:

```
---
paths:
  - "tests/**/*.py"
---

# Testing rules

- Tests must not touch the filesystem, stdout, or a real clock.
```

That is the difference between memory and a rule: memory is always loaded; a rule is loaded when it is relevant. Most `CLAUDE.md` files that have gone bad are rules that were never scoped.

@-imports in `CLAUDE.md` resolve at launch, not lazily — importing a large document costs you its whole size on every session, so import small things.

6. Permissions and the trust boundary

Lab 06. Write a deny list for a Flask app with a live API key in `.env`.

Three ingredients together are what make an agent dangerous. Any two are fine:

1. access to **private data**,

2. exposure to **untrusted content**,
3. the ability to **communicate outward**.

Your editor session usually has all three: your repo, a dependency's README or a web page it fetched, and `curl` or `git push`. Deny rules are how you break the set.

Deny is the only list that is not a convenience. An allow list you get wrong costs you a prompt. A deny list you get wrong costs you a secret. So spend your attention there, and cover the variants — `Read(.env)` does not match `.env.local`, `.env.production` or `.env.staging`, which hold the same secrets.

```
{
  "permissions": {
    "deny": ["Read(.env)", "Read(.env.*)", "Bash(git push *)"],
    "allow": ["Bash(python3 -m pytest *)"]
  }
}
```

Note that `allowed-tools` in a skill or subagent **grants**; it does not restrict. The fence is `permissions`.

Say the boundary out loud as well. Settings are the fence; a sentence in the conversation — *don't touch anything under `migrations/`, and don't push* — is what stops it proposing work you will have to decline five times. Both, not either.

A prompt worth keeping: make it classify the work before it does the work. *Is this a one-line fix, a contained change, or something that touches the data model?* You get the blast radius before you get the diff, and you can stop at the answer. (Ch. 5 has his version, along with the same feature written at three levels of trust.)

7. Skills and hooks

Lab 07. Write a skill that reports coverage gaps, and a hook that formats Python.

The distinction decides which one you should be writing:

- a **skill** is a procedure the agent reaches for **when the situation matches**;
- a **hook** is code that runs **whether or not anyone reached for anything**.

"Run the formatter after every edit" is a hook. "Here is how we investigate a coverage gap" is a skill. Getting this backwards produces a skill nobody invokes, or a hook that fires on work it has no business touching.

The description is the entire interface

Only a skill's frontmatter is loaded at startup; the body is read after the skill has already been chosen. So the description has to carry both **what it does** and **when to use it**, phrased the way

someone would actually ask — *what is untested, where are the coverage holes, is this good enough to open a pull request.*

Two failures, both common:

- A description that says only what the skill does. It never fires, because nothing in it matches how the question gets asked.
- **An unquoted colon.** `description: Reports coverage: gaps and risks` is a nested mapping in YAML. Your description silently truncates at `Reports coverage` and the skill mysteriously stops matching. Quote it.

Test a skill by **not** naming it. If it does not fire on the natural question, the description is the bug, not the body.

Hooks fail in the other direction

A hook runs on every matching tool call, on every machine, including the one that does not have your linter installed. So:

- **exit 0 when the tool is missing.** A hook that errors because `ruff` is absent turns every edit into a failure for someone who never asked for the check.
- **PostToolUse cannot block an edit** — it fires after the write has happened. If you want to stop something, you need `PreToolUse` and exit code 2.
- Ask to see both files before they are written. You are installing code into your own loop.

8. Where the leverage actually is

No lab. A discussion, and the reason the numbering skips 08.

By this point you have a tool that edits files, remembers your conventions, respects a fence and runs procedures. The instinct is to reach for more automation. The better question is where the time actually goes.

It is not typing. It was never typing. It goes into **deciding what to build, finding out how the existing thing works**, and **establishing that a change is correct**. An agentic tool helps enormously with the middle one, somewhat with the third, and not at all with the first — and the first is the one that determines whether the work mattered.

So the leverage is in the parts that are cheap to write and expensive to skip: the spec (§10), the review (§11), and the orientation (§12). Everything else is speed on work you had already decided to do.

9. Subagents and parallel work

Lab 09. Review a change with a subagent you wrote.

A subagent is a **fresh context with a narrow job and a reporting contract**.

The fresh context is the point. A reviewer that watched you write the code has already been convinced by your reasoning — it read the argument for the approach before it read the approach. One that starts cold sees the diff the way a colleague does.

Four things a subagent file needs:

- **frontmatter** with a `name` and a `description` that says *when to delegate to it*. Same rule as a skill: this is what gets matched.
- **a tool list**. An unrestricted subagent is just another session. A reviewer gets read and search tools and nothing that writes. The narrowing is what makes it trustworthy enough to run unattended.
- **standards to review against**, not "review this code".
- **a reporting contract**: what comes back and in what shape. Without it you get whatever it felt like summarising, and you cannot act on a summary.

Invoke it through its interface. If you have to re-explain the job every time, the file is not earning its place.

On running several at once: read-only lenses parallelise freely — a coverage reviewer and a naming reviewer over the same diff cannot collide, and the second routinely sees what the first did not. Writers need **disjoint ownership and a stop condition**, or you get two agents editing the same file and one of them losing. (Ch. 8 has a full worked anti-example of exactly that.)

Two lenses is not two opinions. It is two coverage areas, and neither covers everything — so the last step of the lab is saying what the review missed.

10. From a prompt to a pull request

Lab 10. Write a spec an agent can be held to.

The move that makes this work is **narrowing first**. A backlog item is not a task; it is a heading. Before anything gets written, cut it to one slice with an obvious boundary — and get the alternatives named out loud so the cut is a decision rather than the first idea.

Then the spec. The thing that distinguishes a spec from a task description:

- **postconditions**, not steps. *After this change, an invalid saved index resolves to line 0* — not *add a validation function*. Steps are your design smuggled into the request; postconditions leave the design open and still let you check the result.

- **invariants**, stated as invariants. *Behaviour unchanged. No new dependency. Public API unchanged.*
- **an explicit out-of-scope list**. This is the one people skip and then relitigate in review.

Write invariants knowing §4's finding: they will be obeyed literally, including in the cases where obeying them defeats the purpose. So pair them with a door — *if a constraint makes this awkward, say so rather than working around it.*

The payoff is that when the result is wrong, you can point at the postcondition it violated instead of rewriting the task from scratch. That is the difference between a correction and a restart.

11. Review and recovery

Lab 11. Break something on purpose. See whether the review names the layer.

Review the verifier before you review the change. If the suite is green, the first question is whether the tests still test anything — §4 is the reason. Ask for the two lists (*what changed, what you verified*), and read the second.

Review by risk, not by line count. A 400-line change that adds a module is usually safer than a 4-line change to a boundary condition in shared code. Ask what a future session will be missing — the answer is a list of the things that lived in this conversation and nowhere else, and it is also your commit message.

Diagnose before editing. When something breaks: read the failure, state the likely cause, *then* fix. Jump straight to the fix and you get a repair that never names the assumption that broke, so the same bug comes back in a different shape.

Then decide whether the failure earns a guardrail. Not every bug does. A test, a rule in `CLAUDE.md`, a deny rule, a hook — each has an ongoing cost, and a guardrail added in irritation is paid for by everyone forever. The question is whether this failure will recur, not whether it was annoying.

The lab is deliberately arranged so that a review can *miss* the defect. That is data, not a bug in the exercise. A review that names the layer a defect lives in is useful even when it does not name the line.

12. Landing in a codebase you did not write

Lab 12. `legacy-service`: fifteen modules, five years of history, four tests, and several things that are not what they look like.

Every other lab happens in an app you built four minutes earlier. This is the honest one, and it is the most common thing you will do with an agentic tool in your first month.

Ask for a map, read-only. Four requirements, and the fourth is the one people leave out:

1. what it does,
2. **where execution starts** — the single most useful fact about an unfamiliar codebase, and the one a README most often has wrong,
3. which files to read first, so you get a *reading order* rather than a summary,
4. **where it is guessing versus where the code is explicit.**

In your own project you would notice a confident wrong answer. In a foreign repository you have no such reflex, so make it mark the seam. (Ch. 4 has his orientation prompt; the lab asks you to write yours from these four.)

Then mine the history, which the map will not do for you:

```
Look through the git history for <module> and summarise how its API came to be.  
What decisions does the history explain that the code does not?
```

Scoped to one module, because "summarise the history" of a five-year repo gets you a shrug. *How its API came to be* asks for a sequence rather than a summary. And *what the history explains that the code does not* is the point — you are mining for intent that never made it into a comment. A five-line commit message from 2021 can stop you "fixing" something deliberate.

Spot-check the map against the territory. If it says the request flow runs one way, open one file and follow it. If it calls something dead code, `grep` for the symbol before you believe it. In the lab's fixture, one module's name has been a lie since 2022, one "dead" module still has a live import from a cron script, and a comment asserts a rewrite the git history does not contain.

Then write down only what you verified, deleting the merely plausible. That file is the artifact — orientation is expensive, and paying for it twice is pure waste.

13. When not to use it

Lab 13. Write your stop list, and pick one metric.

The honest boundaries, from the material rather than from caution:

- **Where you cannot check the result.** The tool's failure mode is confident and plausible. If you are not equipped to tell a right answer from a convincing one — an unfamiliar domain, a language you do not read — you are not reviewing, you are agreeing.
- **Where the cost of being wrong is asymmetric.** Migrations, money, deletions, anything outward-facing. Not because it will get it wrong, but because you will not catch it in time when it does.

- **Where the work is the thinking.** If the hard part is deciding what should exist, generating three plausible versions of the wrong thing is not progress.
- **Where the context does not fit.** A change that depends on knowledge spread across a system nobody has written down is a research task first. Do §12, then come back.

The failure mode that actually costs teams is none of those. It is **decision fatigue**: fifty small approvals in an afternoon, each individually reasonable, and by the fortieth you are approving without reading. The defences are structural, not moral — permissions so the boring approvals never reach you, sessions short enough that you are still paying attention at the end, and a habit of reading the diff for the changes you did not ask for rather than the ones you did.

And pick one metric you will actually watch. Not lines generated. Something like: *how often does a change come back in review for something the tests should have caught?* If that number is going up, the speed is not real.

A closing note on the trade

This lab set is deliberately incomplete in one specific way: where a prompt is Kousen's, it is described rather than reproduced, and you write your own from the requirements. That is worse for you in the short run and better in the long run, because the skill being taught is not *having* the prompt. It is knowing which four things the prompt has to do.

If you own the book, read it alongside these. If you do not, the requirements in each brief are enough to write something that works.